

Unidad 1

Almacenamiento información en ficheros

1	Clases para la gestión de ficheros y directorios.....	3
	1.1 Clase File (java.io)	3
	Ejemplo de uso:	3
	Crear un directorio:.....	3
	Listar archivos de un directorio:.....	3
	1.2. Clases Path, Paths y Files (java.nio.file)	4
	Ejemplo de uso:	4
	Crear un directorio:.....	4
	Listar archivos de un directorio:.....	5
	Operaciones habituales con <i>Files</i>	5
	1.3 Clases FileReader y FileWriter	6
2	Ventajas y los inconvenientes de las distintas formas de acceso.	6
	2.1 Acceso Secuencial	7
	Ejemplo: Lectura secuencial de un fichero línea por línea.....	7
	Ejemplo: Escritura secuencial en un fichero de texto.....	7
	Ventajas del acceso secuencial (implícitas en el código):.....	7
	Inconvenientes:	7
	2.2 Acceso Aleatorio (Random Access)	8
	Ejemplo: Escritura aleatoria	8
	Ejemplo: Lectura aleatoria	8
	Ventajas del acceso aleatorio:	8
	Inconvenientes:	8
3	Recuperar información almacenada en ficheros.....	9
	3.1 Recuperar datos de un fichero de texto	9
	Ejemplo: Leer líneas desde un fichero de texto	9
	Ejemplo: Leer texto y procesarlo (campos separados por comas).....	9
	3.2 Recuperar datos de un fichero binario.....	10
	Ejemplo: Lectura con DataInputStream	10
	3.3 Recuperar objetos desde un fichero (Serialización)	11
	Paso 1: Definir una clase serializable	11
	Paso 2: Leer objetos serializados	11
	Consideraciones generales al recuperar datos	12
4	Almacenar información en ficheros.....	12
	4.1 Almacenar información en ficheros de texto	12
	Escribir texto línea por línea.....	12

	Escribir datos separados por comas (CSV).....	13
	4.2 Almacenar información en ficheros binarios.....	13
	Escribir datos binarios (int, double, boolean, etc.)	13
	4.3 Almacenar objetos (Serialización).....	14
	Paso 1: Crear una clase serializable.....	14
	Paso 2: Guardar objetos en un fichero	14
	4.4 Consideraciones importantes al almacenar información	14
5	Conversiones entre diferentes formatos de ficheros.	15
	5.1 Convertir un fichero de texto (CSV) a binario	15
	Código: CSV → Binario	16
	5.2 Convertir un fichero binario a texto (para visualizarlo)	16
	Código: Binario → Texto.....	16
	5.3 Convertir objetos serializados a texto (para exportar).....	17
	Código: Objetos serializados → Texto	17
	5.4 Convertir texto a objetos y serializarlos (Texto → Objetos serializados)	17
	Código: Texto CSV → Serialización de objetos	17
	5.5 Consideraciones comunes en conversiones	18
6	Se han previsto y gestionado las excepciones.....	18
	6.1 Estructura básica para capturar excepciones	19
	6.2 Uso de try-with-resources para cerrar flujos automáticamente.....	19
	Ejemplo: Lectura segura de un fichero	19
	6.3 Tratamiento de fin de fichero (EOF)	20
	Ejemplo: Lectura de objetos serializados con control de fin de fichero	20
	6.4 Validaciones antes de operar con ficheros	20
	Ejemplo: Comprobar si un fichero existe y es legible	20
	6.5 Evitar errores comunes	21
7	Se han probado y documentado las aplicaciones desarrolladas.....	21
	7.1 Pruebas del funcionamiento	21
	Ejemplo: Método de prueba para guardar y recuperar objetos.....	21
	Casos típicos de prueba	22
	7.2 Uso de salidas de depuración (temporal).....	22
	7.3 Documentación del código	23
	Ejemplo: Documentación con comentarios.....	23
	Ejemplo: Comentarios Javadoc	23
	7.4 Recomendaciones para pruebas y documentación	23
8	Ejercicios	24
	Clases para la gestión de ficheros y directorios.....	24
	Ventajas e inconvenientes de las distintas formas de acceso.....	24
	Recuperar información almacenada en ficheros.....	25
	Almacenar información en ficheros	25
	Conversiones entre diferentes formatos de ficheros.....	26
	Se han previsto y gestionado las excepciones	26
	Se han probado y documentado las aplicaciones desarrolladas.....	27

1 Clases para la gestión de ficheros y directorios.

En programación, especialmente en lenguajes como **Java**, la gestión de ficheros y directorios es una tarea común que permite trabajar con datos persistentes. Esto significa que los datos se almacenan en el sistema de archivos y no se pierden al cerrar el programa.

A continuación, se presentan las clases más utilizadas para la gestión de ficheros y directorios, centrándonos en **Java** y **Python**, ya que son lenguajes comunes en ciclos formativos de grado superior.

1.1 Clase File (java.io)

La clase File permite representar rutas (tanto a archivos como a directorios) y realizar operaciones como comprobar su existencia, crear, eliminar, listar, renombrar, etc. Es importante destacar que File **no permite directamente leer o escribir datos**, sino manejar metainformación sobre el archivo o directorio.

Ejemplo de uso:

```
import java.io.File;

public class EjemploFile {
    public static void main(String[] args) {
        File archivo = new File("ejemplo.txt");

        if (archivo.exists()) {
            System.out.println("El archivo existe.");
            System.out.println("Ruta absoluta: " + archivo.getAbsolutePath());
            System.out.println("¿Es un archivo?: " + archivo.isFile());
            System.out.println("¿Es un directorio?: " + archivo.isDirectory());
        } else {
            System.out.println("El archivo no existe.");
        }
    }
}
```

Crear un directorio:

```
File carpeta = new File("nuevaCarpeta");
if (carpeta.mkdir()) {
    System.out.println("Directorio creado correctamente.");
}
```

Listar archivos de un directorio:

```
File directorio = new File("C:\\usuarios");
String[] archivos = directorio.list();

for (String nombreArchivo : archivos) {
    System.out.println(nombreArchivo);
}
```

1.2. Clases Path, Paths y Files (java.nio.file)

A partir de Java 7, Java incorpora una nueva API para trabajar con archivos y directorios en el paquete `java.nio.file`. Las clases principales son:

- **Path**: representa la ruta de un archivo o directorio.
- **Paths**: proporciona métodos para crear objetos `Path` a partir de una ruta.
- **Files**: permite realizar operaciones sobre archivos y directorios, como comprobar si existen, crear, eliminar, copiar, mover o listar.

A diferencia de `File`, esta API ofrece una forma más completa y flexible de trabajar con el sistema de archivos.

Ejemplo de uso:

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class EjemploFiles {
    public static void main(String[] args) {
        Path archivo = Paths.get("ejemplo.txt");
        if (Files.exists(archivo)) {
            System.out.println("El archivo existe.");
            System.out.println("Ruta absoluta: " + archivo.toAbsolutePath());
            System.out.println("¿Es un archivo?: " + Files.isRegularFile(archivo));
            System.out.println("¿Es un directorio?: " + Files.isDirectory(archivo));
        } else {
            System.out.println("El archivo no existe.");
        }
    }
}
```

En este ejemplo, `Paths.get()` crea un objeto `Path` que representa la ruta `ejemplo.txt`.

El objeto `Path` representa **la ubicación** del archivo o directorio, mientras que la clase `Files` proporciona los métodos necesarios para realizar operaciones sobre él.

Crear un directorio:

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class CrearDirectorio {
    public static void main(String[] args) {
        Path carpeta = Paths.get("nuevaCarpeta");
        try {
            Files.createDirectory(carpeta);
            System.out.println("Directorio creado correctamente.");
        } catch (IOException e) {
            System.out.println("No se ha podido crear el directorio.");
        }
    }
}
```

Listar archivos de un directorio:

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.stream.Stream;

public class ListarDirectorio {

    public static void main(String[] args) {

        Path directorio = Paths.get("C:\\\\usuarios");

        try (Stream<Path> archivos = Files.list(directorio)) {

            archivos.forEach(archivo ->
                System.out.println(archivo.getFileName())
            );

        } catch (IOException e) {
            System.out.println("No se puede acceder al directorio.");
        }

    }
}
```

Operaciones habituales con *Files*

Método	Descripción
<code>Files.exists(path)</code>	Comprueba si existe.
<code>Files.isRegularFile(path)</code>	Comprueba si es un archivo.
<code>Files.isDirectory(path)</code>	Comprueba si es un directorio.
<code>Files.createFile(path)</code>	Crea un archivo vacío.
<code>Files.createDirectory(path)</code>	Crea un directorio.
<code>Files.createDirectories(path)</code>	Crea un directorio y sus directorios padre.
<code>Files.delete(path)</code>	Elimina un archivo o directorio.
<code>Files.copy(origen, destino)</code>	Copia un archivo o directorio.
<code>Files.move(origen, destino)</code>	Mueve o renombra un archivo o directorio.
<code>Files.list(path)</code>	Obtiene el contenido de un directorio.

1.3 Clases FileReader y FileWriter

Estas clases permiten **leer** y **escribir** caracteres en ficheros de texto.

```
import java.io.FileWriter;
import java.io.FileReader;
import java.io.IOException;

public class EscrituraLecturaTexto {
    public static void main(String[] args) {
        try {
            FileWriter escritor = new FileWriter("datos.txt");
            escritor.write("Hola, mundo!");
            escritor.close();

            FileReader lector = new FileReader("datos.txt");
            int c;
            while ((c = lector.read()) != -1) {
                System.out.print((char) c);
            }
            lector.close();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

2 Ventajas y los inconvenientes de las distintas formas de acceso.

En Java, existen **diferentes formas de acceder a un fichero**, y cada una de ellas está pensada para un tipo de operación o necesidad específica. Estas formas de acceso se pueden clasificar principalmente en dos categorías:

- **Acceso secuencial**
- **Acceso aleatorio (random access)**

Ambas tienen sus ventajas e inconvenientes según el tipo de fichero, la cantidad de datos y la operación que se desea realizar.

2.1 Acceso Secuencial

En el acceso secuencial, los datos se leen o escriben en orden, desde el principio hasta el final del fichero. Es la forma más simple de acceso, y suele usarse en ficheros de texto o cuando se procesan registros de forma lineal.

Ejemplo: Lectura secuencial de un fichero línea por línea

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LecturaSecuencial {
    public static void main(String[] args) {
        try (BufferedReader lector = new BufferedReader(new FileReader("datos.txt"))) {
            String linea;
            while ((linea = lector.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Ejemplo: Escritura secuencial en un fichero de texto

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.BufferedWriter;

public class EscrituraSecuencial {
    public static void main(String[] args) {
        BufferedWriter escritor;
        try (escritor = new BufferedWriter(new FileWriter("datos.txt", true))) {
            escritor.write("Nueva línea de texto");
            escritor.newLine();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Ventajas del acceso secuencial (implícitas en el código):

- Código más simple y legible.
- Ideal para procesar todos los registros de un fichero.
- Bajo consumo de recursos cuando se procesan archivos pequeños o medianos.

Inconvenientes:

- No se puede acceder directamente a un registro específico sin leer todos los anteriores.
- Ineficiente con archivos muy grandes si solo se necesita una parte concreta del contenido.

2.2 Acceso Aleatorio (Random Access)

Este tipo de acceso permite leer o escribir datos en cualquier parte del fichero, sin necesidad de recorrerlo desde el inicio. En Java se realiza usando la clase *RandomAccessFile*.

Es muy útil cuando se trabaja con ficheros binarios o cuando se necesita acceder a registros de tamaño fijo (por ejemplo, una base de datos sencilla en fichero plano).

Ejemplo: Escritura aleatoria

```
import java.io.RandomAccessFile;
import java.io.IOException;

public class EscrituraAleatoria {
    public static void main(String[] args) {
        try (RandomAccessFile raf = new RandomAccessFile("archivo.bin", "rw")) {
            raf.seek(0); // Posición inicial
            raf.writeUTF("Registro 1");

            raf.seek(100); // Posición 100 del fichero
            raf.writeUTF("Registro 2");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Ejemplo: Lectura aleatoria

```
import java.io.RandomAccessFile;
import java.io.IOException;

public class LecturaAleatoria {
    public static void main(String[] args) {
        try (RandomAccessFile raf = new RandomAccessFile("archivo.bin", "r")) {
            raf.seek(100); // Leer desde la posición 100
            String registro = raf.readUTF();
            System.out.println("Contenido leído: " + registro);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Ventajas del acceso aleatorio:

- Permite acceder directamente a cualquier parte del fichero sin recorrerlo completo.
- Muy eficiente en archivos grandes con registros de tamaño fijo.
- Ideal para simulaciones de bases de datos simples sin usar un motor como SQLite o MySQL.

Inconvenientes:

- Requiere mayor planificación y control de posiciones en el fichero.
- Menor legibilidad y mayor complejidad del código.
- Puede generar errores si no se gestiona correctamente el tamaño y posición de los datos.

3 Recuperar información almacenada en ficheros.

Recuperar información almacenada en ficheros implica **leer los datos previamente guardados** para su procesamiento, visualización o análisis. En Java, la forma de hacerlo dependerá del **tipo de fichero** que estemos leyendo:

- Ficheros de texto (formato plano)
- Ficheros binarios (datos primitivos, objetos, etc.)
- Ficheros serializados (objetos Java completos)

Cada tipo de fichero requiere clases y métodos específicos para recuperar correctamente la información almacenada.

▮ 3.1 Recuperar datos de un fichero de texto

Los ficheros de texto son los más comunes. Contienen datos legibles por humanos y normalmente se accede a ellos línea por línea.

Ejemplo: Leer líneas desde un fichero de texto

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerTexto {
    public static void main(String[] args) {
        try (BufferedReader lector = new BufferedReader(new FileReader("alumnos.txt")))
        {
            String linea;
            while ((linea = lector.readLine()) != null) {
                System.out.println("Línea leída: " + linea);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Ejemplo: Leer texto y procesarlo (campos separados por comas)

// Contenido del fichero: Juan,20,Aprobado

```
import java.io.*;

public class LeerCamposSeparados {
    public static void main(String[] args) {
        try (BufferedReader lector = new BufferedReader(new FileReader("resultados.txt"))) {
            String linea;
            while ((linea = lector.readLine()) != null) {
                String[] campos = linea.split(",");
                System.out.println("Nombre: " + campos[0]);
                System.out.println("Edad: " + campos[1]);
                System.out.println("Estado: " + campos[2]);
            }
        }
    }
}
```

```
        System.out.println("---");
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
```

▮ 3.2 Recuperar datos de un fichero binario

Los ficheros binarios permiten guardar datos en formato no legible, lo que resulta más eficiente en espacio y velocidad. En Java se utilizan clases como `DataInputStream` y `RandomAccessFile`.

Ejemplo: Lectura con `DataInputStream`

```
import java.io.*;

public class LeerBinario {
    public static void main(String[] args) {
        try (DataInputStream entrada = new DataInputStream(new FileInputStream("datos.bin"))) {
            while (entrada.available() > 0) {
                int id = entrada.readInt();
                double nota = entrada.readDouble();
                System.out.println("ID: " + id + ", Nota: " + nota);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Este ejemplo asume que los datos fueron guardados antes con `DataOutputStream` escribiendo un `int` seguido de un `double`.

3.3 Recuperar objetos desde un fichero (Serialización)

Java permite guardar objetos completos en un fichero mediante **serialización**. Para recuperar esos objetos, se usa `ObjectInputStream`.

Paso 1: Definir una clase serializable

```
import java.io.Serializable;

public class Alumno implements Serializable {
    private String nombre;
    private int edad;

    public Alumno(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String toString() {
        return "Alumno: " + nombre + ", Edad: " + edad;
    }
}
```

Paso 2: Leer objetos serializados

```
import java.io.*;

public class LeerObjetos {
    public static void main(String[] args) {
        try (ObjectInputStream entrada = new ObjectInputStream(new FileInputStream("alumnos.dat"))) {
            Alumno alumno;
            while (true) {
                try {
                    alumno = (Alumno) entrada.readObject();
                    System.out.println(alumno);
                } catch (EOFException e) {
                    break; // Fin del fichero
                }
            }
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Este código recupera objetos `Alumno` previamente escritos con `ObjectOutputStream`.

Consideraciones generales al recuperar datos

- Siempre se deben **gestionar las excepciones**, especialmente `IOException`, `EOFException` y `ClassNotFoundException`.
- Es importante **cerrar los flujos de entrada** (manualmente o con `try-with-resources`).
- En ficheros binarios y serializados, el **orden de lectura debe coincidir con el orden de escritura**.
- Se recomienda validar la existencia del fichero antes de intentar leerlo.

4 Almacenar información en ficheros.

Almacenar información en ficheros consiste en **escribir datos desde un programa Java hacia el disco**, de forma que puedan ser recuperados más adelante. Existen diferentes formas de almacenar información, dependiendo del **tipo de fichero** y del **formato de los datos**:

- Ficheros de texto (planos)
- Ficheros binarios
- Ficheros con objetos serializados

Cada uno tiene sus características, usos típicos y clases específicas para trabajar en Java.

▮ 4.1 Almacenar información en ficheros de texto

Los ficheros de texto son los más utilizados para almacenar datos simples en formato legible (como logs, configuraciones, CSV, etc.).

Escribir texto línea por línea

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class EscribirTexto {
    public static void main(String[] args) {
        try (BufferedWriter escritor = new BufferedWriter(new FileWriter("salida.txt"))) {
            escritor.write("Primera línea");
            escritor.newLine();
            escritor.write("Segunda línea");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
}  
}  
}
```

FileWriter permite escribir caracteres. El segundo parámetro puede ser true si se desea **añadir** al final del archivo en lugar de sobrescribirlo.

Escribir datos separados por comas (CSV)

```
import java.io.FileWriter;  
import java.io.PrintWriter;  
import java.io.IOException;  
  
public class EscribirCSV {  
    public static void main(String[] args) {  
        try (PrintWriter pw = new PrintWriter(new FileWriter("alumnos.csv"))) {  
            pw.println("Nombre,Edad,Nota");  
            pw.println("Lucía,21,8.5");  
            pw.println("Mario,22,9.1");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

4.2 Almacenar información en ficheros binarios

Los ficheros binarios permiten guardar datos en formato no legible, lo que mejora el rendimiento y ocupa menos espacio.

Escribir datos binarios (int, double, boolean, etc.)

```
import java.io.DataOutputStream;  
import java.io.FileOutputStream;  
import java.io.IOException;  
  
public class EscribirBinario {  
    public static void main(String[] args) {  
        DataOutputStream salida;  
        try (salida = new DataOutputStream(new FileOutputStream("datos.bin"))) {  
            salida.writeInt(101);  
            salida.writeDouble(9.75);  
            salida.writeBoolean(true);  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Estos datos se deben recuperar con la clase DataInputStream en el mismo orden en que se escribieron.

❏ 4.3 Almacenar objetos (Serialización)

Java permite almacenar objetos completos en un fichero si la clase implementa la interfaz `Serializable`. Esto se conoce como **serialización de objetos**.

Paso 1: Crear una clase serializable

```
import java.io.Serializable;

public class Alumno implements Serializable {
    private String nombre;
    private int edad;

    public Alumno(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String toString() {
        return nombre + " (" + edad + " años)";
    }
}
```

Paso 2: Guardar objetos en un fichero

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

public class GuardarObjetos {
    public static void main(String[] args) {
        Alumno a1 = new Alumno("Laura", 20);
        Alumno a2 = new Alumno("Carlos", 22);

        ObjectOutputStream salida;
        try (salida = new ObjectOutputStream(new FileOutputStream("alumnos.dat"))) {
            salida.writeObject(a1);
            salida.writeObject(a2);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

El fichero `alumnos.dat` contendrá los objetos serializados y podrá recuperarse luego con `ObjectInputStream`.

❏ 4.4 Consideraciones importantes al almacenar información

- Siempre se debe **cerrar el flujo de salida** para garantizar que los datos se escriben correctamente. Usar `try-with-resources` es una buena práctica.
- En binarios y objetos, el **orden de escritura debe respetarse** al momento de leer.

- En escritura de objetos serializados, cualquier cambio en la clase original puede impedir la recuperación de objetos previamente guardados.
- Para grandes cantidades de datos o estructuras complejas, conviene considerar el uso de bases de datos, pero los ficheros son adecuados para configuraciones, logs, backups, etc.

5 Conversiones entre diferentes formatos de ficheros.

En muchas aplicaciones es necesario **convertir datos de un formato de fichero a otro**, por ejemplo:

- De **texto plano a binario**
- De **CSV a JSON**
- De **ficheros serializados** a texto legible
- O incluso de **XML** a objetos Java (o viceversa)

Este tipo de conversiones permite adaptar los datos para distintos propósitos: interoperabilidad con otras aplicaciones, exportación de informes, reducción de tamaño o mejora de rendimiento.

A continuación, se explican distintos escenarios comunes de conversión en Java con sus respectivos ejemplos.

▮ 5.1 Convertir un fichero de texto (CSV) a binario

Supongamos que tenemos un fichero CSV con información de personas:

nombre,edad
Ana,23
Luis,30

Queremos convertirlo a un fichero binario donde cada registro se guarde con un String y un int.

Código: CSV → Binario

```
import java.io.*;

public class CSVaBinario {
    public static void main(String[] args) {
        try {
            BufferedReader lector = new BufferedReader(new FileReader("personas.csv"));
            DataOutputStream salida = new DataOutputStream(new FileOutputStream("personas.bin"))
        ) {
            String linea;
            lector.readLine(); // Saltar cabecera
            while ((linea = lector.readLine()) != null) {
                String[] campos = linea.split(",");
                String nombre = campos[0];
                int edad = Integer.parseInt(campos[1]);

                salida.writeUTF(nombre);
                salida.writeInt(edad);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

▮ 5.2 Convertir un fichero binario a texto (para visualizarlo)

Código: Binario → Texto

```
import java.io.*;

public class BinarioATexto {
    public static void main(String[] args) {
        try {
            DataInputStream entrada = new DataInputStream(new FileInputStream("personas.bin"));
            PrintWriter salida = new PrintWriter(new FileWriter("personas_convertido.txt"))
        ) {
            while (entrada.available() > 0) {
                String nombre = entrada.readUTF();
                int edad = entrada.readInt();

                salida.println(nombre + "," + edad);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

▮ 5.3 Convertir objetos serializados a texto (para exportar)

Si tienes un fichero con objetos serializados, puedes leerlos y exportar sus datos a un fichero de texto plano o CSV.

Código: Objetos serializados → Texto

```
import java.io.*;

public class ObjetosATexto {
    public static void main(String[] args) {
        try {
            ObjectInputStream entrada = new ObjectInputStream(new FileInputStream("alumnos.dat"));
            PrintWriter salida = new PrintWriter(new FileWriter("alumnos.txt"))
        } {
            while (true) {
                try {
                    Alumno alumno = (Alumno) entrada.readObject();
                    salida.println(alumno.getNombre() + "," + alumno.getEdad());
                } catch (EOFException e) {
                    break;
                }
            }
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Nota: Se asume que la clase Alumno tiene métodos getNombre() y getEdad().

▮ 5.4 Convertir texto a objetos y serializarlos (Texto → Objetos serializados)

A veces se necesita cargar datos desde un fichero de texto y almacenarlos como objetos Java serializados para mayor control y eficiencia.

Código: Texto CSV → Serialización de objetos

```
import java.io.*;
import java.util.ArrayList;

public class TextoAObjetos {
    public static void main(String[] args) {
        ArrayList<Alumno> lista = new ArrayList<>();

        try (BufferedReader lector = new BufferedReader(new FileReader("alumnos.csv"))) {
            String linea;
            lector.readLine(); // Saltar cabecera
            while ((linea = lector.readLine()) != null) {
                String[] campos = linea.split(",");
                String nombre = campos[0];
                int edad = Integer.parseInt(campos[1]);
                lista.add(new Alumno(nombre, edad));
            }
        }
    }
}
```

```

        } catch (IOException e) {
            e.printStackTrace();
        }

        ObjectOutputStream salida;
        try (salida = new ObjectOutputStream(new FileOutputStream("alumnos_serializados.dat"))){
            for (Alumno a : lista) {
                salida.writeObject(a);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

▮ 5.5 Consideraciones comunes en conversiones

- Asegúrate de conocer el **formato original** del fichero antes de intentar leerlo.
- En conversiones texto ↔ binario, es obligatorio respetar el **orden y tipo de los datos**.
- Las conversiones pueden implicar **pérdida de formato** si se cambia a un fichero más simple.
- Se recomienda validar los datos antes de escribirlos en el nuevo formato para evitar errores de lectura posteriores.

6 Se han previsto y gestionado las excepciones.

Algunas de las clases de excepción más comunes en operaciones con ficheros son:

- `IOException`: Error general de entrada/salida.
- `FileNotFoundException`: El fichero no existe o no se puede abrir.
- `EOFException`: Fin de fichero alcanzado inesperadamente.
- `ClassNotFoundException`: Al leer un objeto serializado, no se encuentra la clase correspondiente.
- `SecurityException`: Falta de permisos para acceder al fichero.
- `NumberFormatException`: Al convertir texto a números si el formato es incorrecto.

▮ 6.1 Estructura básica para capturar excepciones

```
try {
    // Código que puede lanzar excepciones
} catch (IOException e) {
    System.out.println("Error de E/S: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Error general: " + e.getMessage());
}
```

Se recomienda capturar primero las excepciones más específicas.

▮ 6.2 Uso de try-with-resources para cerrar flujos automáticamente

Una de las mejores prácticas en Java desde la versión 7 es el uso de try-with-resources, que permite cerrar automáticamente los flujos abiertos (como `FileReader`, `BufferedWriter`, etc.).

Ejemplo: Lectura segura de un fichero

```
import java.io.*;

public class LecturaSegura {
    public static void main(String[] args) {
        try (BufferedReader lector = new BufferedReader(new FileReader("entrada.txt")))
        {
            String linea;
            while ((linea = lector.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (FileNotFoundException e) {
            System.out.println("El fichero no se encontró.");
        } catch (IOException e) {
            System.out.println("Error de lectura: " + e.getMessage());
        }
    }
}
```

No es necesario cerrar manualmente `BufferedReader`; se cierra automáticamente al salir del bloque `try`.

▮ 6.3 Tratamiento de fin de fichero (EOF)

Al leer ficheros binarios o con objetos serializados, es común que se lance una excepción al llegar al final del archivo. Esto no es necesariamente un error, sino una señal de que ya no hay más datos.

Ejemplo: Lectura de objetos serializados con control de fin de fichero

```
import java.io.*;

public class LeerObjetosSeguro {
    public static void main(String[] args) {
        try (ObjectInputStream entrada = new ObjectInputStream(new FileInputStream("datos.dat"))) {
            while (true) {
                try {
                    Alumno a = (Alumno) entrada.readObject();
                    System.out.println(a);
                } catch (EOFException e) {
                    break; // Fin del fichero
                }
            }
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Aquí se captura `EOFException` para terminar el bucle de lectura, lo cual es una práctica estándar en Java.

▮ 6.4 Validaciones antes de operar con ficheros

Aunque Java lanza excepciones cuando algo falla, es buena práctica **validar previamente** ciertas condiciones para evitar errores innecesarios.

Ejemplo: Comprobar si un fichero existe y es legible

```
import java.io.File;

public class ValidarFichero {
    public static void main(String[] args) {
        File archivo = new File("config.txt");

        if (archivo.exists() && archivo.canRead()) {
            System.out.println("El archivo está listo para ser leído.");
        } else {
            System.out.println("El archivo no existe o no se puede leer.");
        }
    }
}
```

¶ 6.5 Evitar errores comunes

- **No capturar excepciones en absoluto:** puede hacer que el programa se bloquee.
- **No cerrar flujos de forma segura:** puede causar pérdida de datos o corrupción.
- **No distinguir el tipo de excepción:** impide un tratamiento más preciso del error.
- **No informar adecuadamente al usuario:** dificulta la detección y resolución de problemas.

7 Se han probado y documentado las aplicaciones desarrolladas.

Una vez que se ha desarrollado una aplicación que gestiona ficheros (lectura, escritura, conversión, serialización, etc.), es fundamental aplicar dos buenas prácticas esenciales:

1. **Pruebas del funcionamiento de la aplicación**
2. **Documentación del código fuente**

Estas dos prácticas aseguran que la aplicación sea **fiable, mantenible y comprensible** tanto para el desarrollador original como para futuros programadores que trabajen con ese código.

¶ 7.1 Pruebas del funcionamiento

Probar una aplicación significa ejecutar distintos escenarios y verificar que se comporta correctamente en cada uno de ellos. En el contexto de ficheros, esto incluye comprobar:

- Que se puede leer y escribir correctamente.
- Que se detectan los errores (por ejemplo, fichero no existente).
- Que los datos almacenados se recuperan fielmente.
- Que el contenido del fichero final es el esperado.

Ejemplo: Método de prueba para guardar y recuperar objetos

```
public class PruebaSerializacion {
    public static void main(String[] args) {
        Alumno alumno = new Alumno("Paula", 25);
        String fichero = "alumno_test.dat";

        // Guardar objeto
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(fichero)))
        {
            oos.writeObject(alumno);
            System.out.println("Objeto serializado correctamente.");
        } catch (IOException e) {
            System.err.println("Error al guardar el objeto: " + e.getMessage());
        }
    }
}
```

```

    }

    // Leer objeto
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(fichero))) {
        Alumno recuperado = (Alumno) ois.readObject();
        System.out.println("Objeto recuperado: " + recuperado);
    } catch (IOException | ClassNotFoundException e) {
        System.err.println("Error al leer el objeto: " + e.getMessage());
    }
}
}

```

Este tipo de prueba ayuda a verificar que la serialización y deserialización funcionan correctamente en un caso concreto.

Casos típicos de prueba

Al desarrollar aplicaciones que trabajan con ficheros, conviene definir **casos de prueba** que incluyan:

- Fichero inexistente
- Fichero vacío
- Datos mal formateados (por ejemplo, en CSV)
- Fichero con permisos restringidos
- Fichero demasiado grande
- Escritura concurrente (si aplica)

Estos casos ayudan a anticipar errores en entornos reales.

7.2 Uso de salidas de depuración (temporal)

Durante las pruebas, se pueden añadir **mensajes de depuración** para facilitar el seguimiento del flujo del programa:

```

System.out.println("Abriendo el fichero...");
System.out.println("Línea leída: " + linea);
System.out.println("Guardando objeto: " + objeto);

    En aplicaciones reales, estos mensajes se sustituyen por logs usando
    bibliotecas como java.util.logging o log4j.

```

7.3 Documentación del código

La documentación es clave para entender el funcionamiento de la aplicación y facilitar su mantenimiento. En Java, se suele documentar mediante:

- **Comentarios en el código**
- **Comentarios Javadoc**
- **Ficheros README o informes técnicos (en proyectos más grandes)**

Ejemplo: Documentación con comentarios

```
// Guardar un objeto Alumno en un fichero
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("alumno.dat"))) {
    oos.writeObject(alumno); // Serializa el objeto
} catch (IOException e) {
    e.printStackTrace(); // Imprime error si no se puede guardar
}
```

Ejemplo: Comentarios Javadoc

```
/**
 * Guarda un objeto Alumno en un fichero.
 * @param alumno Objeto Alumno a guardar
 * @param nombreFichero Ruta del fichero de destino
 */
public void guardarAlumno(Alumno alumno, String nombreFichero) {
    // implementación...
}
```

Javadoc permite generar documentación HTML automáticamente a partir de estos comentarios con el comando javadoc.

7.4 Recomendaciones para pruebas y documentación

- Comprobar siempre los datos generados en disco (ver si el contenido es correcto).
- Utilizar nombres de ficheros claros y temporales en pruebas (test_data.txt, alumno_test.dat, etc.).
- Separar el código de prueba del código principal cuando sea posible.
- Añadir comentarios explicativos para cada bloque relevante del código.
- Incluir instrucciones de uso si la aplicación será usada por otras personas.

8 Ejercicios

▮ Clases para la gestión de ficheros y directorios

1. Crear una utilidad que liste todos los archivos y subdirectorios de una ruta introducida por el usuario.
2. Mostrar todos los ficheros .txt de un directorio y sus fechas de modificación.
3. Pedir al usuario una ruta y comprobar si es un fichero, un directorio o no existe.
4. Crear un programa que cree un árbol de directorios a partir de una estructura definida por código.
5. Eliminar de forma recursiva todos los ficheros de un directorio que tengan más de 7 días.
6. Copiar todos los ficheros de un tipo específico de una carpeta a otra.
7. Comprobar el tamaño total de todos los ficheros en una carpeta y mostrarlo en MB.
8. Crear un programa que renombre todos los archivos de una carpeta aplicando un formato específico (por ejemplo: IMG_001.jpg, IMG_002.jpg, etc.).
9. Buscar en un directorio y subdirectorios todos los archivos que contengan una palabra clave en su nombre.
10. Crear un historial de los ficheros creados, modificados o eliminados en una carpeta (simulación de un sistema de monitoreo).

▮

▮ Ventajas e inconvenientes de las distintas formas de acceso

1. Leer un fichero grande línea por línea (acceso secuencial) y contar cuántas líneas contienen una palabra clave.
2. Crear una clase Registro de tamaño fijo y guardarla en un fichero binario usando acceso aleatorio.
3. Modificar un registro en una posición concreta de un fichero sin reescribir el fichero completo.
4. Comparar el tiempo de lectura secuencial vs aleatoria en un fichero con 10.000 registros de tamaño fijo.
5. Crear un programa que permita insertar, modificar y eliminar registros en un fichero binario con acceso aleatorio.
6. Escribir un índice con las posiciones de los registros en un fichero para acceder a ellos más rápidamente.
7. Diseñar una pequeña base de datos de empleados usando acceso aleatorio. Permitir búsqueda por ID.

8. **Implementar una herramienta que lea un fichero de texto, pero que permita saltar a una línea específica usando un índice generado previamente.**
9. **Simular un sistema de reservas con acceso aleatorio a un fichero binario de asientos de avión.**
10. **Guardar logs secuenciales y luego extraer solo los de un rango de fechas, comparando eficiencia frente a binarios.**

▮

▮ **Recuperar información almacenada en ficheros**

1. **Leer un fichero de texto con datos de usuarios (nombre, edad, correo) y mostrarlos por consola.**
2. **Leer un fichero CSV y cargar los datos en una lista de objetos.**
3. **Leer datos binarios y convertirlos a un formato legible en consola.**
4. **Leer un fichero de objetos serializados y mostrarlos ordenados por uno de sus atributos.**
5. **Leer un fichero JSON (usando librerías externas como Gson) y convertirlo en objetos Java.**
6. **Leer un fichero con errores de formato y capturar las excepciones sin detener el programa.**
7. **Cargar un fichero binario de productos y permitir buscarlos por nombre o código.**
8. **Leer un fichero de configuración (config.properties) y mostrar los valores en consola.**
9. **Leer un fichero de texto con líneas que contienen expresiones matemáticas y evaluarlas.**
10. **Cargar un fichero que contiene logs de una aplicación, agruparlos por tipo (INFO, ERROR, DEBUG, etc.).**

▮

▮ **Almacenar información en ficheros**

1. **Guardar una lista de objetos Producto en un fichero de texto en formato CSV.**
2. **Guardar registros con DataOutputStream y asegurarte de que el formato sea legible al leer.**
3. **Serializar una lista de objetos Alumno y guardarla en un fichero .dat.**
4. **Crear un fichero log.txt que guarde todos los errores que ocurran en tiempo de ejecución.**
5. **Escribir en un fichero la información ingresada por el usuario desde la consola hasta que escriba “FIN”.**

6. **Guardar información en un fichero binario pero con compresión usando GZIPOutputStream.**
7. **Escribir un fichero de texto que contenga una tabla formateada (como una factura o listado con columnas).**
8. **Serializar objetos con atributos anidados (por ejemplo, Alumno que contiene Direccion).**
9. **Guardar un fichero CSV con separador personalizado (; en vez de ,).**
10. **Simular una base de datos simple: guardar, modificar y eliminar datos persistentes usando serialización.**

▮

▮ **Conversiones entre diferentes formatos de ficheros**

1. **Convertir un fichero CSV de productos a un fichero binario.**
2. **Convertir un fichero binario a un fichero de texto legible para exportación.**
3. **Cargar objetos desde un fichero .dat y exportarlos a CSV.**
4. **Convertir un fichero de texto con estructura XML a objetos Java y viceversa (con JAXB).**
5. **Leer un fichero JSON y exportarlo como CSV (requiere uso de librerías).**
6. **Convertir un listado de objetos serializados a texto tabulado (formato tipo Excel).**
7. **Crear un programa que lea un CSV de usuarios y lo convierta a un fichero JSON.**
8. **Hacer backup automático de un fichero de texto en formato binario.**
9. **Convertir un fichero de texto con datos en formato personalizado a objetos y luego a binario.**
10. **Leer registros de un fichero binario y exportarlos a una página HTML con formato de tabla.**

▮

▮ **Se han previsto y gestionado las excepciones**

1. **Crear una aplicación que intente leer un fichero que no existe y capture adecuadamente la excepción.**
2. **Simular la apertura de un fichero protegido (sin permisos) y capturar SecurityException.**
3. **Leer un fichero mal formateado y capturar NumberFormatException sin que se caiga el programa.**
4. **Leer objetos de un fichero .dat con fin de fichero controlado con EOFException.**
5. **Crear un programa que intente escribir en una carpeta inexistente y capture el error.**

6. **Hacer una clase de utilidad que reciba rutas y devuelva errores específicos personalizados.**
7. **Comprobar si el fichero está en uso (bloqueado) y manejar la excepción.**
8. **Implementar un logger que guarde todas las excepciones en un fichero errores.log.**
9. **Crear un validador que antes de leer un fichero, compruebe su existencia y permisos.**
10. **Realizar operaciones de escritura concurrente en un fichero y capturar posibles errores.**

▮

▮ Se han probado y documentado las aplicaciones desarrolladas

1. **Crear un programa de ejemplo que lea y escriba un fichero, documentar cada método con Javadoc.**
2. **Simular una batería de pruebas manuales: introducir errores de entrada y comprobar respuestas.**
3. **Escribir un README (en texto plano o Markdown) que explique cómo ejecutar y probar la aplicación.**
4. **Documentar el proceso de conversión de ficheros con pasos y resultados esperados.**
5. **Escribir métodos testX() dentro de una clase de pruebas con diferentes escenarios de ficheros.**
6. **Probar qué ocurre al leer un fichero vacío y documentar el comportamiento esperado.**
7. **Realizar pruebas de rendimiento en lectura secuencial vs aleatoria y documentar los resultados.**
8. **Generar logs de cada operación realizada y analizar si las salidas son coherentes.**
9. **Añadir mensajes de depuración en todos los métodos de gestión de ficheros y revisar el flujo.**
10. **Aplicar assert en diferentes puntos del código para verificar estados intermedios y documentar los casos.**